Espectroscopía.

La espectroscopía es el estudio de la **interacción entre la radiación electromagnética y la materia**. Se basa en el análisis del espectro de luz, ondas electromagnéticas u otras partículas (como en la espectrometría de masas) que son *emitidas*, *absorbidas* o *dispersadas* por una sustancia. Esta técnica es fundamental en múltiples disciplinas científicas, como la química, la física y la astronomía, ya que permite **identificar materiales, analizar estructuras moleculares y estudiar la composición de objetos lejanos**, como estrellas o galaxias.

Existen diversas modalidades de espectroscopía, entre las que destacan:

† **Espectroscopía de absorción:** Analiza las longitudes de onda absorbidas por una muestra al incidir sobre ella radiación.

† **Espectroscopía de emisión:** Estudia la radiación emitida por átomos o moléculas que han sido previamente excitadas.

† **Espectroscopía de fluorescencia:** Mide la luz reemitida por una muestra tras haber sido excitada con radiación, generalmente en un proceso de relajación electrónica.

† **Espectroscopía Raman:** Se basa en la **dispersión inelástica de la luz**, lo que permite obtener información sobre vibraciones moleculares y modos normales.

† **Espectroscopía Brillouin:** Similar a la espectroscopía Raman, pero centrada en la interacción de la luz con ondas acústicas y **fonones de baja frecuencia** en medios condensados.

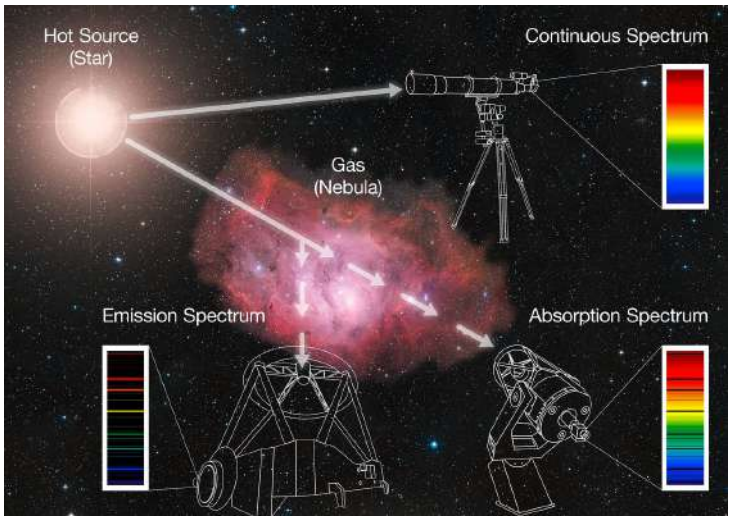
† **Espectroscopía infrarroja (IR):** Estudia la absorción de radiación infrarroja por parte de **enlaces químicos**, proporcionando información sobre las vibraciones moleculares y la estructura de la muestra.

† **Espectroscopía de fotoelectrones (XPS/UPS):** Basada en el **efecto fotoeléctrico**, permite estudiar la composición química y los niveles de energía electrónica en la superficie de un material.

† **Espectroscopía de fotoluminiscencia (PL):** Mide la emisión de luz posterior a la **excitación óptica** de un material, proporcionando información sobre transiciones electrónicas y estados excitónicos. Este es el tipo de espectroscopía realizada en este trabajo.

Cada una de estas técnicas proporciona un tipo distinto de información espectral, que puede manifestarse en forma de líneas o bandas características en un **espectro**. Las

diferencias entre los procesos de absorción y emisión se reflejan directamente en la forma del espectro obtenido, como se ilustra en la siguiente figura.



Comparación entre espectroscopía de absorción y de emisión en un contexto estelar. [European Southern Observatory, sf].

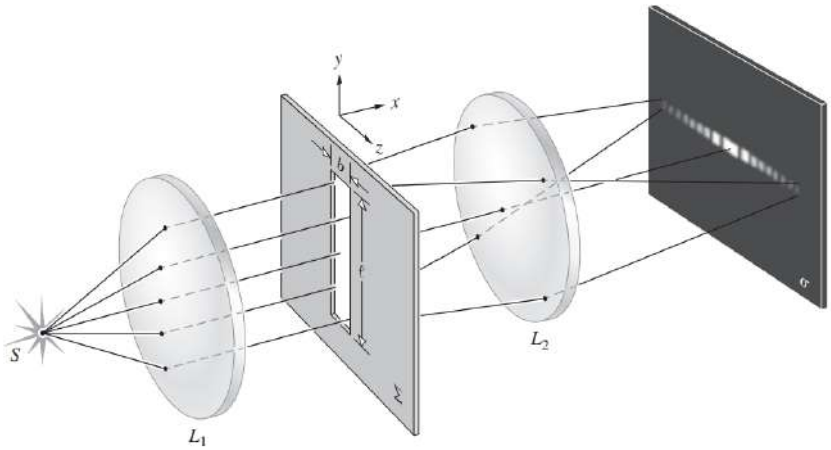
Los dispositivos empleados para registrar estos espectros se denominan **espectrómetros**. Su uso con fines científicos se remonta a mediados del siglo XIX, con las contribuciones clave de Gustav Kirchhoff y Robert Bunsen al desarrollo de la espectroscopía moderna.

Difracción de Fraunhofer

La **difracción** es un fenómeno característico de las ondas, consistente en la desviación de su trayectoria al encontrarse con un obstáculo o una abertura. En el caso de la luz, la difracción da lugar a **patrones de interferencia** que pueden ser analizados para obtener información sobre la geometría del sistema o la naturaleza de la onda incidente.

La **aproximación de Fraunhofer** describe el régimen de difracción en el que tanto la fuente de luz como la pantalla de observación están situadas a distancias suficientemente grandes respecto al obstáculo, o bien cuando se emplean lentes colimadoras que permiten **aproximar las ondas incidentes como planas**. Esta condición permite un tratamiento matemático más sencillo, ya que los frentes de onda pueden considerarse planas y la propagación de la luz se describe mediante **integrales de Fourier**.

Cuando una onda plana incide sobre una rendija estrecha, cada punto de esta abertura actúa como un emisor secundario de ondas esféricas, según el **principio de Huygens-Fresnel**. La interferencia de estas ondas secundarias en el plano lejano origina un patrón de intensidad característico, cuya forma depende de la anchura de la rendija y de la longitud de onda utilizada. En este contexto, la rendija actúa como un modulador espacial de fase y amplitud, y **el patrón observado no es más que la transformada de Fourier del perfil de apertura**.

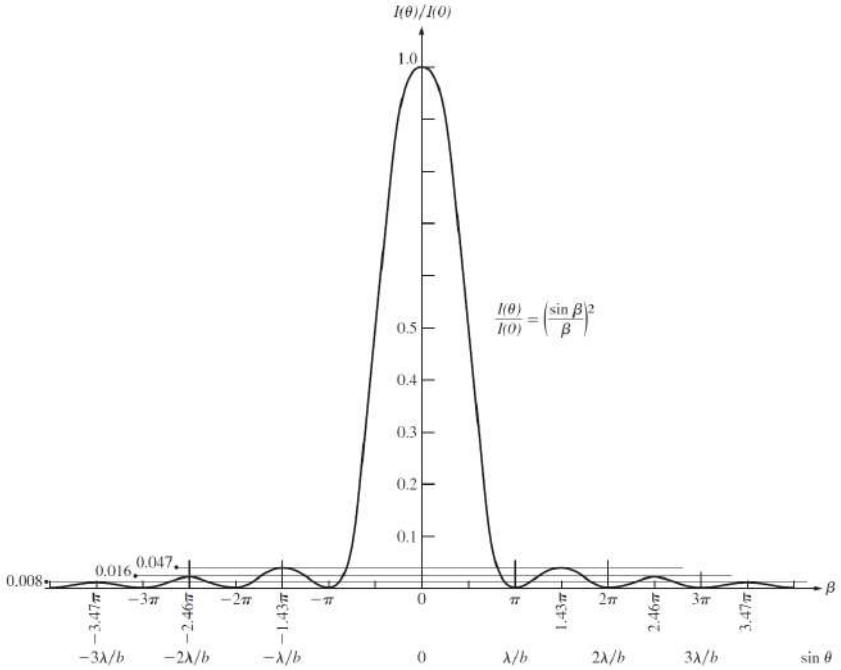


Difracción de Fraunhofer en una rendija rectangular. [Hecht, 2017, p. 466]

Para una rendija de anchura b, iluminada por una onda monocromática de longitud de onda λ, se puede demostrar [Jenkins and White, 2001, pp. 316–319] que la distribución angular de la intensidad en un punto P, situado a un ángulo θ respecto a la normal de la rendija, está dada por:

$$I(\theta) \propto \left(\frac{\sin\beta}{\beta}\right)^2; \quad \beta = \frac{\pi b \sin\theta}{\lambda} \tag{1}$$

Esta función describe un **máximo central intenso y simétrico** acompañado de máximos secundarios de menor intensidad.



Intensidad de difracción de Fraunhofer normalizada. [Hecht, 2017, p. 472]

A partir de la ecuación 1 se deduce que los **mínimos** de intensidad (interferencias destructivas) se producen en las posiciones angulares θ que satisfacen:

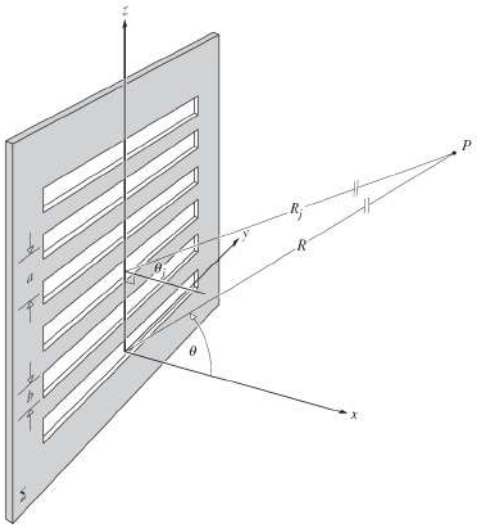
$$b\sin\theta = m\lambda; \quad m \in \mathbb{Z} \setminus \{0\} \tag{2}$$

Por otro lado, los **máximos** de intensidad (interferencias constructivas) aparecen aproximadamente en los puntos intermedios entre los mínimos sucesivos. Sin embargo, solo el máximo central es perfectamente definido, ya que los máximos secundarios tienen intensidades decrecientes y no se encuentran en posiciones exactas derivadas de una condición analítica simple.

El número entero m se denomina **orden de difracción**, y está relacionado con la periodicidad de los mínimos del patrón. Es importante notar que, dado que el ángulo de difracción depende de la longitud de onda, una fuente policromática generará máximos en posiciones distintas para cada componente espectral. Este principio permite, mediante el uso de rendijas o redes de difracción, **separar espacialmente diferentes longitudes de onda**, sentando las bases físicas de la espectroscopía.

Redes de difracción.

Una red de difracción es un dispositivo óptico compuesto por un conjunto regular de rendijas, ranuras o surcos paralelos, espaciados a una distancia (d) comparable con la longitud de onda de la luz incidente. Su función es dispersar la luz en diferentes direcciones según su longitud de onda, debido al fenómeno de interferencia constructiva entre las ondas difractadas. Se utilizan ampliamente en espectroscopía y en dispositivos como monocromadores y espectrómetros para separar distintas longitudes de onda.



Representación de una red de difracción. [Hecht, 2017, p. 481]

Una forma de resolver este sistema es usando el método de suma de amplitudes complejas [Jenkins and White, 2001, pp. 357 - 359]. Para ello se denota por a la magnitud de las amplitudes aportadas por las N rendijas individuales de la red de difracción. La fase cambia un paso δ de una rendija a la siguiente, por lo que la amplitud compleja es la suma de la serie geométrica:

$$Ae^{i\theta} = \sum_{j=0}^{N-1} ae^{ij\delta} = a\sum_{j=0}^{N-1} \left(e^{i\delta}\right)^j = a\frac{1-e^{iN\delta}}{1-e^{i\delta}} \tag{3}$$

Multiplicando a ambos lados de la igualdad por el complejo conjugado de la expresión se obtiene:

$$A^2 = a^2 \frac{\left(1 - e^{iN\delta}\right)\left(1 - e^{-iN\delta}\right)}{\left(1 - e^{i\delta}\right)\left(1 - e^{-i\delta}\right)} = a^2 \frac{1 - cosN\delta}{1 - cos\delta} = a^2 \frac{sin^2 N\frac{\delta}{2}}{sin^2\frac{\delta}{2}} \equiv a^2 \frac{sin^2 N\gamma}{sin^2\gamma} \tag{4}$$

Donde se ha usado la relación trigonométrica:

$$1 - cos\alpha = 2sin^2\frac{\alpha}{2} \tag{5}$$

Por otro lado, de forma geométrica se puede llegar a la expresión:

$$\gamma = \frac{\delta}{2} = \frac{\pi d sin\theta}{\lambda} \tag{6}$$

Juntándolo todo, se obtiene una expresión para la intensidad:

$$I \approx A^2 = A_0^2 \frac{sin^2\beta}{\beta^2} \frac{sin^2 N\gamma}{sin^2\gamma} \tag{7}$$

Por supuesto en el límite de una única rendija ($N = 1$) se recupera la expresión 1. Además, no es muy complicado llegar a una expresión para las posiciones angulares de los principales máximos (mucho más intensos, son los usados para hacer espectroscopía):

$$m\lambda = d sin\theta \tag{8}$$

Si uno hace un desarrollo más general teniendo en cuenta la posibilidad de que la luz incida la red con un ángulo α llega a una expresión similar conocida como ecuación de *grating* [Jenkins and White, 2001, p. 360]

$$d(sin\theta + sin\alpha) = m\lambda \tag{9}$$

La separación entre 2 colores aumenta con el número de órdenes de magnitud. Para conmesurar este fenómeno se define la **dispersión angular** como el ratio de cambio de ángulo respecto a cambios en la longitud de onda. Por tanto esta dispersión es [Jenkins and White, 2001, p.362]:

$$\frac{\Delta\theta}{\Delta\lambda} = \frac{m}{d cos\theta} \tag{10}$$

De esta expresión se deduce que **las distintas longitudes de onda están más separadas a mayores órdenes**. De hecho, el poder de resolución de dos longitudes de onda separadas $\Delta\lambda$ es:

$$R = \frac{\lambda}{\Delta\lambda} = mN' \tag{11}$$

Donde N' denota el número de rendijas iluminadas. De esto se deduce que para pasos angulares fijos, **para aumentar la resolución del espectrómetro** hay dos opciones. O bien aumentar el orden de difracción lo cual conlleva una incidencia más rasante y por tanto efectos como ondas evanescentes que no se han tenido en cuenta, o bien aumentar el número de rendijas iluminadas lo cual no siempre es trivial.

En la **literatura anglosajona**, se utiliza frecuentemente el término *diffraction grating* para referirse a las redes de difracción. Aunque en muchos contextos se emplean como sinónimos, en algunos textos se hace una distinción sutil: el término ***grating*** suele

referirse de forma más amplia a cualquier elemento óptico con una estructura periódica que provoque difracción, incluyendo tanto redes de transmisión como de reflexión, y no necesariamente limitadas a surcos rectilíneos.

Existen dos tipos principales de redes de difracción: las **redes de transmisión**, donde la luz pasa a través de las rendijas, y las **redes de reflexión**, donde la luz se refleja en los surcos tallados sobre una superficie reflectante. Estas últimas se utilizan habitualmente en instrumentos de espectroscopía de alta resolución, debido a que permiten una **mayor densidad de líneas por milímetro** y una mejor eficiencia en ciertas bandas espectrales. Además, dependiendo del perfil de los surcos (triangulares, sinusoidales, *blazed*, etc.), es posible optimizar la eficiencia de la red para un rango específico de longitudes de onda, lo que se conoce como ***blazing*** de la red.

Monocromadores

Un monocromador es un dispositivo óptico diseñado para **aislar una longitud de onda específica** de una fuente policromática.Esto resulta esencial en técnicas como la espectroscopía, la microscopía espectral o la caracterización de materiales por sus propiedades ópticas.

El principio de funcionamiento se basa en la dispersión angular de la luz según su longitud de onda, realizada mediante un **elemento dispersivo** (típicamente un prisma o una red de difracción), seguida de una **rendija de salida** que actúa como filtro espacial. Ajustando la geometría interna del sistema, se puede seleccionar la componente espectral deseada.

Desde el punto de vista práctico, los monocromadores se clasifican según el elemento dispersivo utilizado:

† **Monocromadores de prisma**: Aprovechan la dependencia del índice de refracción con la longitud de onda (**aberración cromática**), descrita por relaciones como la ecuación de Cauchy o la de Sellmeier. La desviación angular $\theta(\lambda)$ viene dada por las **leyes de la refracción en medios no homogéneos**. Estos monocromadores presentan una dispersión continua pero no lineal, lo que complica su calibración espectral.

† **Monocromadores de red de difracción**: Son los más comunes en instrumentación científica por su alta resolución espectral y linealidad. Utilizan una red de difracción como elemento dispersivo, y la condición para los máximos de intensidad viene dada por la ecuación del *grating*, expresión 9. El sistema óptico suele incorporar espejos colimadores y lentes para asegurar que las condiciones de Fraunhofer se cumplan. Uno de los problemas que presentan es el **solapamiento de órdenes** ($m > 1$), lo que obliga al uso de filtros adicionales para aislar el orden deseado.

† **Monocromadores acusto-ópticos (AOM)**: Basados en **cristales birrefringentes** atravesados por ondas acústicas de radiofrecuencia, que modulan localmente el índice de refracción, provocando una difracción controlada electrónicamente de la luz. Tienen la ventaja de ser totalmente *solid-state* (sin partes móviles) y extremadamente rápidos en la conmutación de longitudes de onda, pero su resolución espectral es limitada y presentan pérdidas ópticas significativas. La eficiencia de difracción en estos dispositivos depende tanto del material como de la frecuencia aplicada y suele limitar el rango útil de trabajo.

Huecos, movilidades y semiconductores dopados.

Teniendo en cuenta la dualidad onda-corpúsculo postulada por Louis de Broglie en 1924, se puede escribir el momento como:

$$\lambda_{DB} = \frac{h}{p} \Leftrightarrow p = \frac{h}{\lambda_{DB}} = \frac{hk}{2\pi} = \hbar k \tag{12}$$

Y de esta manera las expresiones 2.6 se reducen a:

$$p_h = -p_e \Leftrightarrow k_h = -k_e \ ; \quad q_h = -q_e = +e \ ; \quad \varepsilon_h = -\varepsilon_e \tag{13}$$

Los semiconductores se pueden estudiar considerando unos pocos electrones en la banda de conducción y muchos electrones en la banda de valencia o bien **unos pocos electrones en la banda de conducción y unos pocos huecos en la banda de valencia**. Este segundo enfoque facilita mucho las cosas ya que evita resolver problemas a muchos cuerpos. De esta manera, la conductividad total es:

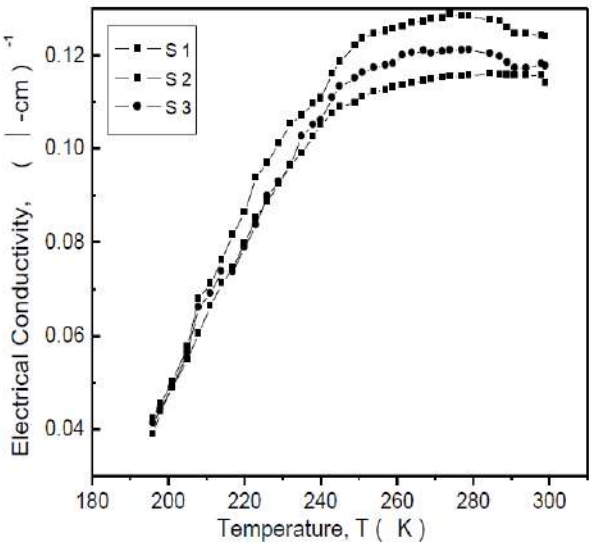
$$\sigma = \sigma_e + \sigma_h = ne\mu_e + pe\mu_h \tag{14}$$

Donde n denota la densidad electrónica (electrones por unidad de volumen) y p la densidad de huecos (huecos por unidad de volumen) y las variables $\mu_e,\ \mu_h$ son la movilidades de electrones y huecos respectivamente. La **movilidad** representa qué tan rápido se mueve un portador de carga (como un electrón o un hueco) dentro de un material cuando se aplica un campo eléctrico. Cuanto mayor es la movilidad, más fácilmente puede desplazarse el portador, lo que se traduce en una mayor conductividad eléctrica. Es una medida de **qué tan libremente pueden moverse los portadores** sin ser frenados por colisiones o interacciones con el entorno del material.

Usualmente, **la movilidad de los huecos suele ser menor que la de los electrones** debido a diferencias en la estructura de bandas del material. Como se ha comentado, los huecos se asocian a la banda de valencia, que presenta una curvatura menos pronunciada y una mayor complejidad en sus estados electrónicos, lo que da lugar a una **respuesta más lenta frente a un campo eléctrico**. Además, los huecos tienden a dispersarse más fácilmente al interactuar con la red cristalina, lo que también contribuye a reducir su movilidad respecto a los electrones en la banda de conducción. Es decir:

$$\tau_h < \tau_e \Rightarrow \mu_h < \mu_e \tag{15}$$

Al aumentar la temperatura (o el potencial químico) se tiene energía suficiente para excitar más electrones formándose más pares electrón-hueco y por tanto dotando al material de más portadores de carga libres. Es decir **los semiconductores aumentan su conductividad con la temperatura**, tal y como se aprecia en la siguiente figura.



Datos empíricos de σ vs T para Si tipo P. [Sharmin et al., 2015, p. 38].

Hasta este punto se ha considerado exclusivamente el comportamiento de **semiconductores intrínsecos**, es decir, materiales en los que las concentraciones de electrones y huecos son iguales, determinadas únicamente por las propiedades intrínsecas del cristal. No obstante, en muchas aplicaciones tecnológicas es deseable alterar esta simetría para incrementar de forma controlada la cantidad de portadores mayoritarios (ya sean electrones o huecos), y con ello modificar la conductividad eléctrica del material. Esta finalidad se logra mediante el proceso conocido como **dopaje**.

El dopaje consiste en introducir **impurezas sustitucionales** en la red cristalina del semiconductor, reemplazando átomos del material base por especies químicas pertenecientes al grupo III (dopantes aceptores) o al grupo V (dopantes donadores) de la tabla periódica. Esta sustitución genera niveles energéticos adicionales dentro del intervalo prohibido (band gap) del material.

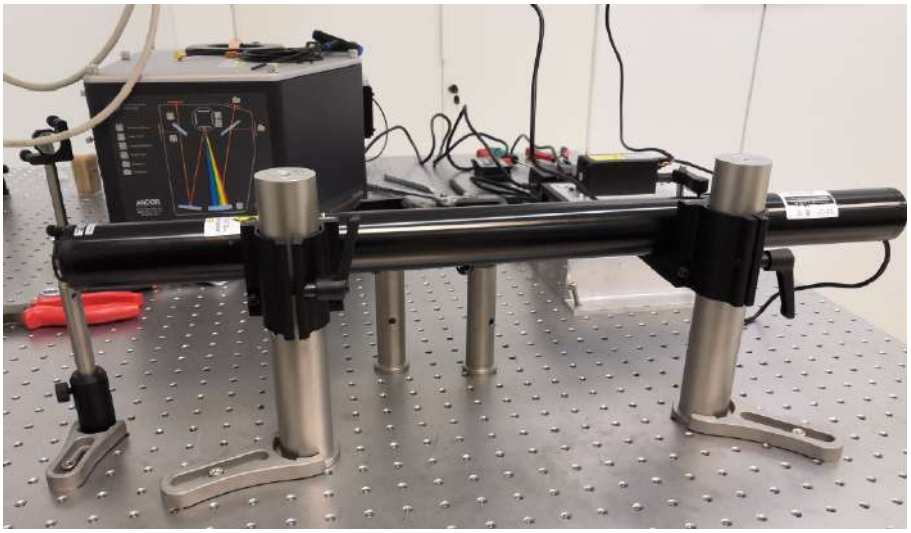
La incorporación de dopantes puede llevarse a cabo mediante procesos de **difusión en fase gaseosa** a altas temperaturas o mediante **implantación iónica**, técnica en la que se bombardea el sólido con iones del elemento dopante. En ambos casos, para minimizar la aparición de defectos estructurales como dislocaciones, las concentraciones de dopaje se mantienen generalmente en el orden de una parte por millón.

Los átomos donadores introducen niveles energéticos discretos ligeramente por debajo de la banda de conducción, típicamente separados por unas pocas centésimas de electrón-voltio (eV). A temperaturas suficientemente altas, los electrones pueden ser excitados térmicamente desde estos niveles hacia la banda de conducción, contribuyendo así a la conductividad eléctrica.

De manera análoga, **los átomos aceptores crean niveles energéticos situados a pequeñas distancias energéticas por encima de la banda de valencia**. Cuando un electrón de dicha banda es promovido hacia uno de estos niveles vacíos, se genera un hueco que actúa como portador positivo, incrementando la conductividad del semiconductor tipo-p. [Callister and Rethwisch, 2015, pp. 520–523]

Partes y funcionamiento de un láser de He-Ne.

Los láseres de helio-neón (He-Ne) son una clase de láseres de gas que emiten luz coherente mediante la **excitación de una mezcla gaseosa** compuesta típicamente por helio y neón, confinada en un tubo sellado a baja presión. Estos dispositivos operan mediante descarga eléctrica, que transfiere energía a los átomos de helio. La línea de emisión más común de estos láseres se encuentra en el rojo ($\lambda \approx 633\ nm$), aunque existen configuraciones que permiten emisiones en otras longitudes de onda.



Láser de He-Ne.

En el contexto de técnicas de μ-PL, los láseres de He-Ne se emplean como fuentes de excitación debido a su **alta estabilidad espectral, buena coherencia espacial y monocromaticidad**. Estas características los hacen especialmente adecuados para la excitación controlada de semiconductores bidimensionales, puntos cuánticos u otras nanoestructuras. En un experimento típico de μ-PL, el haz láser se enfoca mediante un sistema óptico (frecuentemente un objetivo de alta apertura numérica) sobre una región localizada de la muestra, generando pares electrón-hueco que, al recombinarse radiativamente, emiten fotones cuya energía, intensidad y distribución espectral proporcionan **información sobre los estados excitónicos**, defectos, confinamiento cuántico o interacciones con el entorno local.

El uso de láseres He-Ne en esta técnica permite, además, minimizar la generación de calor local o daños estructurales en la muestra, debido a su baja potencia típica ($1-5\ mW$) y a la posibilidad de una excitación no resonante controlada. Esto es particularmente relevante en materiales sensibles como monocapas de dicalcogenuros de metales

de transición (TMDs), donde las propiedades ópticas pueden degradarse bajo condiciones de iluminación intensas o prolongadas.

Su funcionamiento se basa en los principios generales de la física láser: generación de una **inversión de población**, amplificación óptica mediante **emisión estimulada** y realimentación de la radiación mediante un **sistema resonante**.

En este tipo de láser, el **medio activo** está constituido por una mezcla gaseosa de helio (He) y neón (Ne) en proporciones típicas de 5:1 a 10:1, contenida en un tubo capilar sellado a baja presión (del orden de 1 Torr). El sistema es excitado por una **descarga eléctrica continua** de corriente continua (DC), que provoca la ionización parcial del gas y la excitación de los átomos de helio a estados metaestables. La elección de estos dos gases no es arbitraria: existe un acoplamiento eficiente entre ciertos niveles excitados del helio y niveles específicos del neón, lo que permite **transferir energía de forma no radiativa**.

La clave para el funcionamiento del láser He-Ne reside en este mecanismo de transferencia de energía: los átomos de helio excitados colisionan con átomos de neón en su estado fundamental, elevándolos a niveles de energía superiores específicos que se corresponden con los niveles láser del neón. Esta **inversión de población** en los niveles del neón permite que, ante la presencia de un fotón incidente con la energía correspondiente, se desencadene un proceso de emisión estimulada. La cavidad óptica, compuesta por dos espejos enfrentados, uno altamente reflectante y otro parcialmente transmisivo, permite amplificar dicha radiación coherente por múltiples reflexiones y extraer un haz colimado y estable.

A continuación se detallan cada una de las principales partes de este dispositivo y el funcionamiento de cada una de ellas.

Tubo de descarga.

El tubo de descarga constituye el **componente central** del láser de helio-neón, siendo el espacio físico donde se produce la emisión coherente de luz mediante la interacción de la mezcla gaseosa con una descarga eléctrica controlada. Este tubo está fabricado típicamente en **vidrio de borosilicato o cuarzo**, materiales que proporcionan alta resistencia térmica y química, además de transparencia óptica en el rango visible. Está herméticamente sellado y contiene la **mezcla gaseosa a baja presión**.

Las dimensiones internas del tubo suelen estar en un rango estrecho, con diámetros de entre 1 y 3 mm y longitudes que oscilan entre 15 y 50 cm, lo que contribuye a definir una geometría óptima para la propagación del modo transversal fundamental (TEM_{00}) dentro de la cavidad óptica. Esta configuración espacial es fundamental para limitar la divergencia del haz y maximizar la calidad del modo de emisión.

La función principal del tubo es alojar el medio activo y permitir que se genere y mantenga una descarga eléctrica continua a lo largo de su eje longitudinal. En este proceso, una alta tensión aplicada entre electrodos produce una corriente de electrones acelerados que colisionan con los átomos de helio, excitándolos a estados metaestables de larga vida, tales como los niveles 2^3S_1 y 2^1S_0. Posteriormente, estos átomos de helio excitados transfieren su energía a los átomos de neón mediante **colisiones inelásticas resonantes**, elevando electrones neonianos a niveles excitados superiores. Este mecanismo induce una **inversión de población** en los niveles energéticos del neón, condición imprescindible para la amplificación por emisión estimulada. Es fundamental que el tubo mantenga condiciones físicas precisas, como la presión y la pureza de la mezcla gaseosa, para asegurar

una alta tasa de colisiones efectivas y una mínima absorción o pérdidas no deseadas, lo que se traduce en una **ganancia óptica** suficiente para sostener la emisión láser estable y continua.

Este mecanismo de excitación y transferencia de energía puede visualizarse mediante un **diagrama de niveles de energía**, que muestra las transiciones clave implicadas en el proceso láser. En él se representan los niveles metaestables del helio, los niveles excitados del neón y las transiciones radiativas responsables de la emisión a 632,8 nm. La representación gráfica facilita la comprensión del acoplamiento energético entre especies y la ruta seguida hasta alcanzar la emisión estimulada, permitiendo identificar los procesos fundamentales que hacen posible la generación coherente de luz en este tipo de láser.

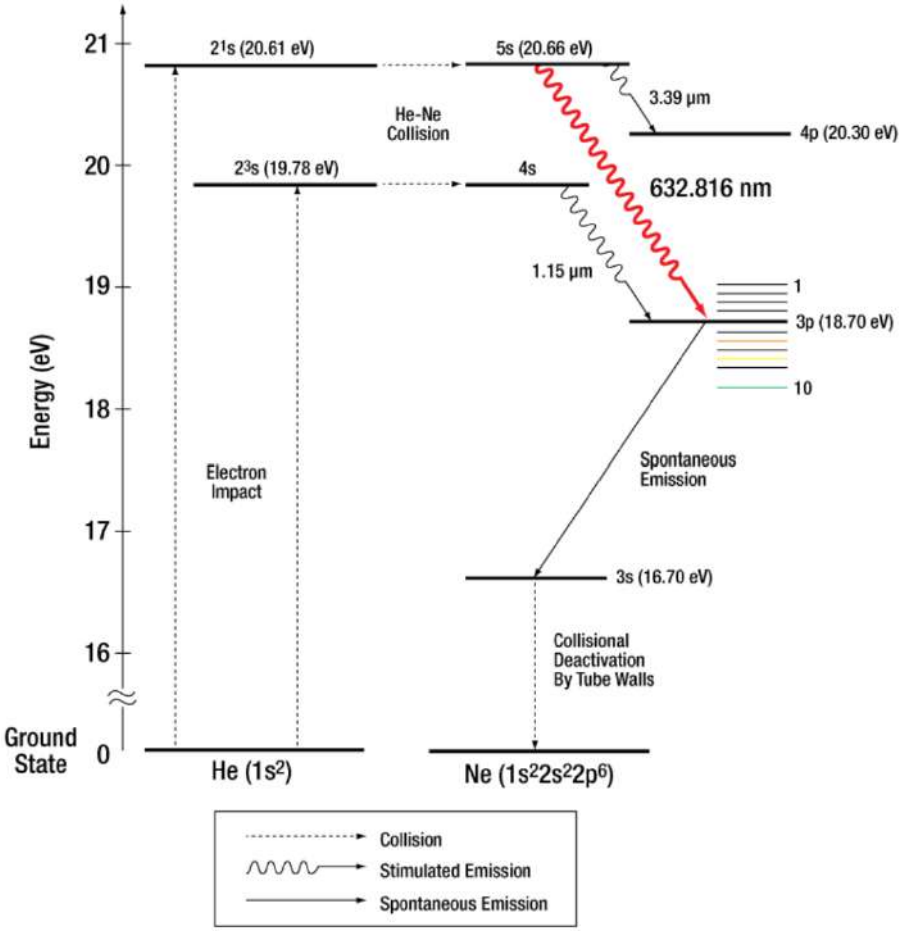


Diagrama de niveles energéticos. [Thorlabs, sf]

Electrodos y sistema de descarga.

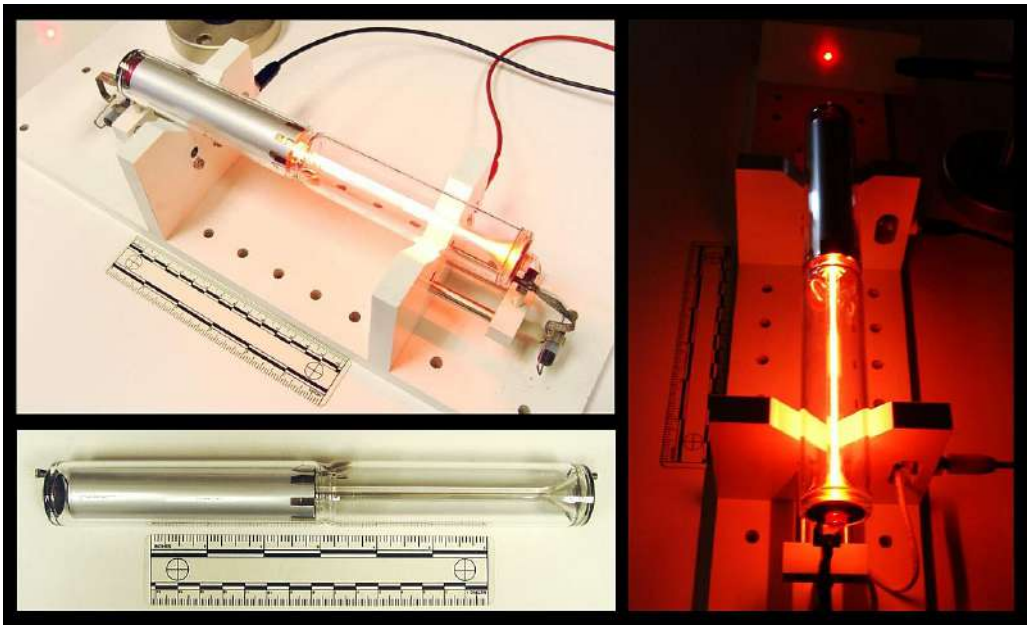
Los láseres de He-Ne se activam mediante una descarga eléctrica de corriente continua aplicada a lo largo del tubo de descarga. Para ello, se emplean dos electrodos metálicos, generalmente de aluminio, acero inoxidable o tungsteno, **ubicados en los extremos del tubo** o incrustados en cámaras auxiliares conectadas al volumen principal por canales capilares. Estos electrodos se conectan a una fuente de alta tensión continua (típicamente entre 1 y 5 kV), que genera un campo eléctrico capaz de ionizar parcialmente la mezcla gaseosa, produciendo un **plasma débilmente ionizado**.

El flujo de electrones acelerados por el campo eléctrico colisiona con los átomos de

helio, excitándolos a estados metaestables sin llegar a ionizarlos completamente. Este proceso es crucial para permitir la posterior transferencia de energía al neón. La estabilidad de la descarga es esencial para mantener una **emisión láser continua y sin fluctuaciones**; por ello, el circuito suele incluir una resistencia limitadora en serie para regular la corriente (del orden de 3 a 10 mA) y evitar arcos eléctricos o inestabilidades. Además, el diseño de los electrodos y su aislamiento deben minimizar efectos como la erosión, el calentamiento localizado y la formación de depósitos, que podrían degradar el rendimiento óptico y eléctrico del dispositivo con el tiempo.

Cavidad óptica resonante.

El sistema óptico resonante del láser de helio-neón está formado por **dos espejos dispuestos coaxialmente** a lo largo del eje del tubo de descarga, constituyendo una cavidad Fabry-Pérot. Uno de ellos es altamente reflectante ($R > 99{,}9\,\%$), mientras que el otro es parcialmente transmisivo ($T \approx 1\,\%$), permitiendo la extracción del haz láser. Esta disposición permite la retroalimentación óptica necesaria para la amplificación de la radiación por emisión estimulada. La longitud de la cavidad y la curvatura de los espejos (planos o cóncavos) se escoge cuidadosamente para favorecer la estabilidad transversal del modo fundamental TEM_{00} y minimizar las pérdidas por difracción.



Interior de un láser de He-Ne. [Experimental Engineering, sf]

Los espejos están recubiertos con **multicapas dieléctricas optimizadas para reflejar selectivamente la longitud de onda** de emisión deseada. El alineamiento preciso de la cavidad es crítico, ya que incluso pequeñas desviaciones angulares pueden impedir la oscilación láser. En configuraciones comerciales, los espejos suelen estar fijados permanentemente mediante técnicas de pegado óptico, mientras que en sistemas experimentales pueden utilizarse monturas ajustables para facilitar la optimización del acoplamiento y la alineación de modos longitudinales y transversales.

Soporte mecánico y aislamiento

El tubo láser y el sistema óptico requieren un **soporte mecánico que garantice la estabilidad estructural y térmica** del conjunto. Para ello, se utilizan carcasas cilíndricas, generalmente de aluminio anodizado, que protegen el tubo de descarga y los espejos de factores externos como vibraciones, corrientes de aire, polvo o variaciones de temperatura. Estas carcasas no solo proporcionan rigidez mecánica, sino que también actúan como **blindaje frente a campos eléctricos** y radiación electromagnética no deseada.

En sistemas comerciales o de laboratorio con requisitos de alta estabilidad, se incluyen elementos adicionales como **monturas antivibración**, soportes ajustables y materiales con baja expansión térmica para mantener la alineación óptica. Además, es frecuente incorporar aislamiento térmico pasivo, e **incluso control activo de temperatura**, con el fin de evitar fluctuaciones en la longitud de onda de emisión debidas a cambios térmicos. Estas precauciones son especialmente relevantes cuando el láser se utiliza como fuente de excitación en experimentos espectroscópicos de alta resolución, donde la estabilidad espacial y espectral del haz es crítica.

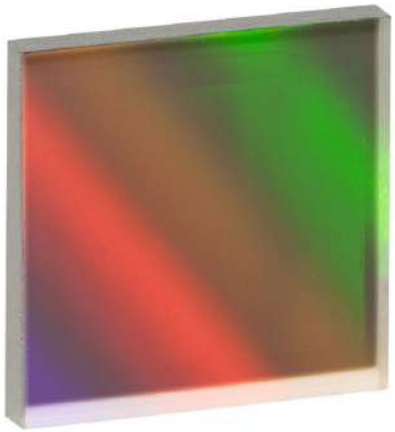
Implementación de un monocromador en transmisión.

Un paso fundamental en el diseño experimental consiste en separar las diferentes longitudes de onda que componen la fuente de luz utilizada. Para ello, se ha implementado un **monocromador** en configuración de transmisión, cuyo principio de funcionamiento se basa en la dispersión angular inducida por una **red de difracción** o *grating*. Este tipo de monocromador consta de tres elementos fundamentales: la red de difracción, un **motor paso a paso** que permite la rotación angular de dicha red en el eje perpendicular al plano de incidencia y un **fotodiodo** fijo encargado de detectar la luz difractada. La rotación controlada de la red permite seleccionar, con alta resolución espectral, la longitud de onda que incide sobre el detector. De este modo, el sistema actúa como un filtro óptico sintonizable, permitiendo analizar de forma selectiva regiones concretas del espectro.

Elementos del monocromador.

† Un ***grating* de transmisión**, modelo GTI25-03A de ThorLabs, centrado en el infrarrojo cercano y con un ángulo de ranura de 31,7°. A diferencia de las redes reflectivas, una red de transmisión permite que la luz pase a través de la misma.

(a) Eficiencia en función de λ.



(b) Fotografía.

Grating de transmisión. [Thorlabs, sf].

Como se puede apreciar en la anterior figura, el modelo GTI25-03A está optimizado para operar en el rango del infrarrojo cercano (NIR), presentando una alta eficiencia de transmisión en esa región del espectro ($\sim 66\,\%$ en torno a 800 nm). Esto lo convierte en el modelo ideal para este proyecto pues se espera el **pico de emisión excitónica en torno**

a $\sim$ 750 nm (para WSe_2). Por otro lado, esta red cuenta con $N = 300$ *líneas por mm* por lo que:

$$d = \frac{1}{N} \simeq 3{,}33\ \mu m \tag{16}$$

Acoplada mecánicamente al eje del motor paso a paso, la rotación de la red permite seleccionar con precisión la longitud de onda que se dirige hacia el detector, funcionando como elemento dispersivo clave en el diseño del monocromador.

† Un **motor NEMA 17**, modelo 42BYGH24-401S, está acoplado mecánicamente a la red de difracción. Este tipo de motor paso a paso contiene un rotor con **dientes imantados** y un estator con múltiples bobinas distribuidas en fases. El funcionamiento se basa en la **activación secuencial de estas bobinas**, lo que genera campos magnéticos que atraen los dientes del rotor, forzándolo a alinearse con el campo resultante.

Cada vez que cambia el estado de las bobinas, el rotor avanza un paso fijo de 1.8°, lo que permite un control preciso de la posición angular sin necesidad de sensores externos. Este motor opera en **modo bipolar**, por lo que requiere una inversión de corriente en cada bobina para funcionar correctamente.

En este modelo, el par de retención es de aproximadamente 0,38 Nm, suficiente para mantener estable la posición de la red incluso frente a pequeñas perturbaciones mecánicas. Además, al combinarse con un controlador como el TB6600, es posible operar el motor en **modo de micropasos** (por ejemplo, hasta 1/16 de paso completo), lo que mejora sustancialmente la resolución angular. Para un paso de 1,8°, el micropaso más pequeño posible es de 0,1125°, lo que permite **escaneos espectrales con una resolución mucho mayor** sin requerir componentes ópticos móviles adicionales.

El movimiento se genera gracias a la interacción entre los polos del rotor y los **campos magnéticos rotatorios** inducidos en el estator, lo que proporciona un par elevado a bajas velocidades. Esto lo hace especialmente adecuado para aplicaciones de posicionamiento preciso, como el ajuste fino de una red de difracción.

(a) Perspectiva 1 del motor.

(b) Perspectiva 2 del motor.

Distintas perspectivas del motor NEMA.

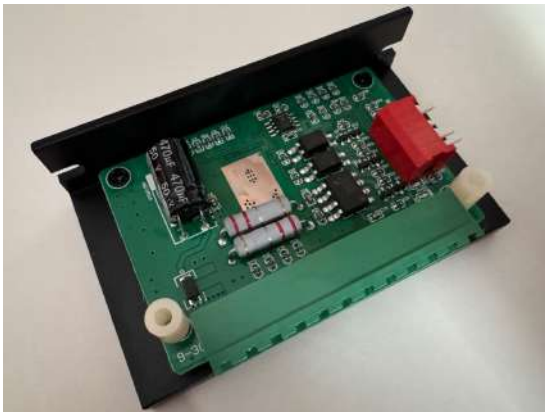
† Un **controlador TB6600** que es el encargado de regular el funcionamiento del motor paso a paso, actuando como interfaz entre la señal digital de control (generada por Arduino) y las bobinas del motor. Su función principal es **gestionar la distribución de corriente** a través de las fases del motor, permitiendo un control preciso del movimiento rotacional.

Internamente, el TB6600 emplea una arquitectura basada en **transistores MOSFET** dispuestos en configuración de puente H doble. Esta topología le permite conmutar la dirección de la corriente en cada fase del motor de forma eficiente y rápida, esencial para el funcionamiento en modo bipolar, donde es necesario invertir la polaridad en cada bobina para generar el par motor adecuado.

Una de las características más relevantes del TB6600 es su capacidad para operar en modo de **micropasos**. Esta funcionalidad permite subdividir cada paso completo del motor (1.8°) en múltiples pasos más pequeños (hasta 1/16 de paso, dependiendo de la configuración), lo que **mejora significativamente la resolución angular** y la suavidad del movimiento. Esto resulta especialmente útil en aplicaciones de posicionamiento fino, como la rotación controlada de una red de difracción.

Además, el controlador incorpora diversos **mecanismos de protección** que garantizan la integridad del sistema en condiciones de operación exigentes. Entre ellos se incluyen la **protección contra sobrecorriente, sobrecalentamiento y cortocircuitos**. Estas funciones aseguran un funcionamiento robusto y estable, incluso bajo cargas variables o en caso de errores de conexión, protegiendo tanto al motor como a la electrónica de control.

(a) Parte externa del controlador.



(b) Parte interna del controlador.

Partes del controlador.

† Un **Arduino UNO R3** que actúa como unidad de control principal del sistema. Este microcontrolador basado en el *ATmega328P* se encarga de generar las señales digitales necesarias para gobernar el controlador TB6600. A través de dos salidas digitales, este microcontrolador envía **pulsos de control**: una señal `STEP` que determina la frecuencia de avance del motor, y otra `DIR` que establece el sentido de rotación.

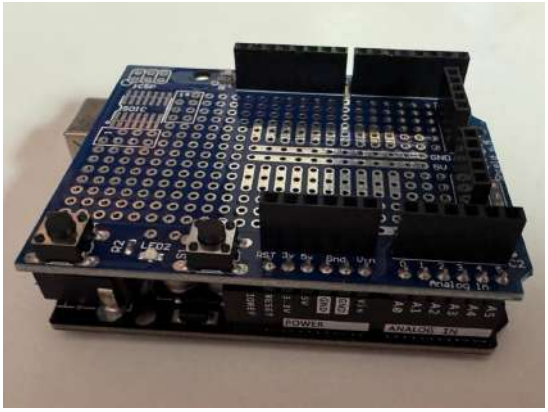
El sistema puede complementarse con entradas externas (potenciómetros, pulsadores o **comunicación serie**), lo que permite modificar en tiempo real parámetros como la posición objetivo o la velocidad de barrido. En concreto en este proyecto se explotará la comunicación serie para controlarlo con un *script* de Python.

Desde un punto de vista técnico, el *ATmega328P* funciona a 16 MHz y dispone de *timers* internos que permiten generar señales de temporización estables, fundamentales para el control por pasos. Estos *timers* pueden ser configurados para producir **trenes de pulsos con precisión temporal** mediante funciones de retardo por software (`delayMicroseconds`) o por interrupciones programadas, dependiendo de los requerimientos de velocidad y precisión.

Dado que el controlador TB6600 requiere niveles lógicos TTL estándar (0-5 V), el Arduino UNO es especialmente adecuado por su **compatibilidad nativa**. El ancho de pulso mínimo aceptado por el TB6600 es del orden de 10 μs, valor fácilmente alcanzable por el Arduino incluso bajo carga, siempre que se gestione correctamente la temporización mediante interrupciones. Además, el consumo típico del Arduino ronda los 50 mA, lo que permite alimentarlo sin problemas desde un puerto USB o una fuente regulada de 5 V. Estas características aseguran una **integración sencilla y eficiente** dentro de sistemas de control optomecánico como el empleado en este proyecto.

Gracias a su arquitectura RISC de 8 bits y al uso de registros específicos para las entradas y salidas digitales, es capaz de responder rápidamente a cambios de estado, garantizando un control fiable incluso en tareas de tiempo real. Además, su bajo consumo y la facilidad de reprogramación lo convierten en una **solución ideal para tareas de automatización ligera**.

(a) Arduino sin *shield* acoplado.



(b) Arduino con *shield* acoplado.

Arduino usado para el monocromador.

† Un **fotodetector de silicio**, modelo APD430A/M de Thorlabs, se encarga de registrar la luz monocromática que emerge del monocromador tras la difracción. Este dispositivo es un fotodetector de tipo APD (*Avalanche Photodiode*), optimizado para el rango espectral de 400 a 1000 nm, lo que lo hace especialmente adecuado para aplicaciones en el visible y el infrarrojo cercano.

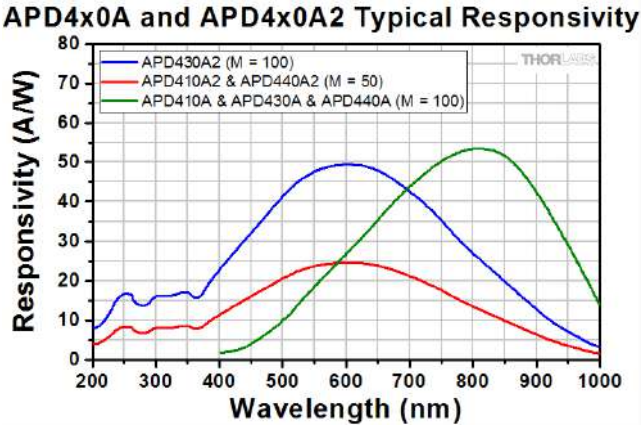
El principio de funcionamiento de un APD se basa en la multiplicación de portadores por avalancha: cuando un fotón incide sobre el **semiconductor**, genera un par electrón-hueco, y bajo un campo eléctrico elevado, estos portadores adquieren suficiente energía para ionizar átomos del material, generando nuevos portadores. Este proceso proporciona una **ganancia interna ajustable** (hasta $\times 50$ en este modelo), lo que permite detectar señales ópticas de baja intensidad con alta sensibilidad y buena relación señal-ruido.

Desde el punto de vista físico, la **eficiencia cuántica** del fotodetector, $\eta(\lambda)$, y su coeficiente de ganancia M dependen críticamente de la energía de los fotones incidentes y del diseño de la región de multiplicación. En este modelo, la respuesta espectral máxima se sitúa en torno a los 800 nm, coincidiendo con el pico de sensibilidad del silicio. Además, la corriente de oscuridad ($I_{\text{dark}} < 10$ nA típicamente) se mantiene lo suficientemente baja como para garantizar **mediciones fiables** incluso en condiciones de baja iluminación, siendo un parámetro clave para reducir el ruido de fondo.

El APD430A/M incorpora una etapa de amplificación transimpedancia y ofrece una salida analógica de tensión proporcional a la intensidad luminosa, alimentándose mediante

fuente externa o vía USB. En este experimento, el fotodetector permite **convertir la señal óptica en una señal eléctrica** que puede visualizarse o registrarse, facilitando el análisis espectral de la fuente.

(a) Fotografía del detector.



(b) Curva de respuesta del detector.

Fotodetector utilizado. [Thorlabs, sf].

Aplicación de Python.

Un componente fundamental de este proyecto es el desarrollo de una aplicación en Python encargada de controlar y automatizar el funcionamiento del monocromador. Esta aplicación incorpora una **interfaz gráfica** (*frontend*) que permite configurar parámetros esenciales como el **tiempo de adquisición** por ángulo (longitud de onda) y los límites angular del barrido. Además, facilita la visualización en tiempo real del espectro adquirido y la exportación sencilla de datos y gráficos al finalizar las mediciones.

Patrones de diseño y programación orientada a objetos

La gestión simultánea de la adquisición y el procesamiento de datos constituye un problema clásico de concurrencia en programación. Para abordarlo, se emplean diversos **patrones de diseño** que optimizan la ejecución desacoplada y eficiente de tareas. Entre estos paradigmas destaca la **arquitectura Lambda**, inicialmente concebida para computación en la nube.

La arquitectura Lambda se basa en la ejecución de **funciones independientes, autocontenidas y activadas por eventos específicos**, favoreciendo así una alta escalabilidad y modularidad sin necesidad de gestionar explícitamente los recursos subyacentes. Cada función se focaliza en una tarea concreta, siguiendo principios de separación de responsabilidades y procesamiento reactivo.

Conceptualmente, este modelo es análogo al uso de **programación concurrente con hilos** (*threads*), donde cada hilo ejecuta una unidad funcional en paralelo al hilo principal, buscando maximizar la capacidad de respuesta y eficiencia del sistema.

Aunque en este proyecto no se implementa una arquitectura Lambda formal, sí se adopta una estrategia equivalente mediante la delegación de tareas específicas a **hilos independientes**. Esto permite mantener una estructura modular y concurrente, garan-

tizando que la interfaz gráfica permanezca receptiva mientras se realiza la adquisición y procesamiento de datos en segundo plano.

Para validar el correcto funcionamiento de la aplicación en fases tempranas, se implementó un **generador de números pseudoaleatorios** que simula las señales del fotodiodo. Esta aproximación simplifica el desarrollo y facilita la depuración antes de la integración con el hardware físico.

Las principales funcionalidades implementadas incluyen:

† Un **histograma** central que se actualiza dinámicamente con cada nueva muestra.

† Un campo de texto para definir el **tiempo de adquisición**, entendido como el intervalo temporal entre capturas consecutivas.

† Un **botón** que permite iniciar, pausar y reiniciar la adquisición.

† Un **botón** para guardar el histograma actual en formatos gráficos estándares (PNG, JPEG).

El diseño de la interfaz se ha implementado siguiendo el paradigma de **programación orientada a objetos** (POO), indispensable para mantener el código modular, escalable y fácil de mantener. En POO, los objetos agrupan atributos y métodos relacionados, encapsulando la lógica y facilitando la reutilización.

En esta aplicación, la clase principal, denominada `Monochromator`, encapsula tanto los elementos gráficos (botones, controles deslizantes, etc.) como la lógica de adquisición, actualización y almacenamiento de datos. La instancia de esta clase representa la aplicación completa, **ejecutándose en un ciclo continuo hasta que el usuario cierra la ventana**.

Dependencias y esquema de comunicación

El desarrollo se realiza en Python 3.12 utilizando librerías robustas que permiten integrar la interfaz gráfica, el procesamiento de datos y la comunicación con el *hardware*. Entre ellas destacan las siguientes:

† `Tkinter`: biblioteca estándar para la creación de interfaces gráficas, que proporciona widgets como ventanas, botones y campos de texto.

† `Matplotlib`: para la generación y actualización dinámica de gráficos, especialmente útil para la visualización en tiempo real del histograma espectral.

† `threading`: módulo nativo para la gestión de hilos, permitiendo la ejecución concurrente de la adquisición de datos y la actualización de la interfaz sin bloqueos.

† `PySerial`: librería que facilita la comunicación serie entre el ordenador y el microcontrolador Arduino UNO, permitiendo el envío de comandos y la recepción de datos en tiempo real.

El flujo de comunicación se organiza de la siguiente manera:

1. El usuario establece los parámetros de adquisición mediante la interfaz gráfica.

2. La aplicación Python envía comandos al Arduino UNO a través del puerto serie, especificando las posiciones angulares (pasos del motor) y el tiempo de integración para cada medida.

3. El Arduino controla el motor paso a paso, ajustando la posición angular de la red de difracción.

4. Paralelamente, la aplicación Python se comunica directamente con una DAC (*data acquisition commponent*) conectada vía USB para adquirir la señal analógica generada por el fotodiodo en cada posición angular.

5. Un hilo dedicado recibe los datos de la DAC, actualiza el histograma en tiempo real y almacena la información para análisis posteriores.

6. El usuario puede detener la adquisición en cualquier momento y exportar los datos y gráficos generados.

Esta arquitectura desacoplada, donde el control mecánico y la adquisición de señal se gestionan mediante dispositivos y protocolos independientes, garantiza un funcionamiento fluido y responsivo, esencial para experimentos de alta precisión y adquisición dinámica de datos.

Código de Python para la GUI (*Grafical User Interface*).

```
import tkinter as tk
from tkinter import filedialog
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg
import threading, time, serial

# === Global Configuration ===
default_fontsize = 20  # Font size for all GUI elements

# === Main Application Class ===
class MonochromatorApp:
    def __init__(self, root):
        self.root = root
        self.root.title('Random Number Generator')
        self.root.geometry('1000x1000')

        # Enable resizable layout
        for i in range(3):
            self.root.columnconfigure(i, weight=1)
        self.root.rowconfigure(1, weight=1)
        self.root.rowconfigure(2, weight=0)

        font_large = ('Arial', default_fontsize)
        font_button = ('Arial', default_fontsize, 'bold')

        # === Acquisition Time Input ===
        self.label_time = tk.Label(root, text='Acquisition time (s):', font=font_large)
        self.label_time.grid(row=0, column=0, sticky='w', padx=10, pady=5)

        self.time_entry = tk.Entry(root, font=font_large, width=15)
        self.time_entry.insert(0, '3')  # Default value
        self.time_entry.grid(row=0, column=1, sticky='ew', padx=10, pady=5)

        # === Start/Stop Button ===
        self.start_button = tk.Button(root, text='Start', command=self.toggle_running, font=font_button)
        self.start_button.grid(row=0, column=2, padx=10, pady=5)

        # === Plot Setup ===
        self.fig, self.ax = plt.subplots(figsize=(8, 5))
```

```
        self.ax.set_facecolor('azure')
        self.ax.set_title('Random Number Histogram', fontsize=default_fontsize, fontweight='bold')
        self.ax.set_xlabel('Value', fontsize=default_fontsize)
        self.ax.set_ylabel('Frequency', fontsize=default_fontsize)
        self.ax.grid(True, color='gray')

        self.canvas = FigureCanvasTkAgg(self.fig, master=root)
        self.canvas.get_tk_widget().grid(row=1, column=0, columnspan=3, sticky='nsew', padx=10, pady=10)

        # === Save Button ===
        self.save_button = tk.Button(root, text='Save', command=self.save_plot, font=font_button)
        self.save_button.grid(row=2, column=1, pady=10)

        # === State Variables ===
        self.running = False
        self.started_once = False
        self.mean = 10  # Mean of generated random numbers

        # === Arduino Serial Connection ===
        try:
            self.arduino = serial.Serial('COM3', 9600, timeout=1)  # Update COM port as needed
            time.sleep(2)  # Wait for Arduino initialization
        except Exception as e:
            print(f'[ERROR] Could not connect to Arduino: {e}')
            self.arduino = None

    def toggle_running(self):
        """Start or stop acquisition."""
        if self.running:
            self.running = False
            self.start_button.config(text='Restart')
        else:
            self.running = True
            self.start_button.config(text='Stop')

            if not self.started_once:
                self.started_once = True

            # Start acquisition in a separate thread
            threading.Thread(target=self.generate_numbers, daemon=True).start()

    def generate_numbers(self):
        """Main acquisition loop: communicates with Arduino and updates histogram."""
        numbers = []

        while self.running:
            try:
                acquisition_time = float(self.time_entry.get())
                if acquisition_time < 0:
                    acquisition_time = 0
            except ValueError:
                acquisition_time = 1  # Fallback to 1 second
```

```python

            # Send step command to Arduino
            if self.arduino:
                try:
                    self.arduino.write(b'S')  # Command Arduino to step
                    line = self.arduino.readline().decode().strip()
                    while line != 'DONE':
                        line = self.arduino.readline().decode().strip()
                except Exception as e:
                    print(f'[ERROR] Arduino communication: {e}')

            # Simulate data acquisition
            time.sleep(acquisition_time)
            numbers.append(np.random.normal(self.mean, 1))
            self.update_plot(numbers)

    def update_plot(self, numbers):
        """Clear and redraw histogram with updated data."""
        self.ax.clear()
        self.ax.grid(True, color='gray', zorder=0)
        self.ax.hist(numbers, bins=20, color='cornflowerblue', edgecolor='k', density=True, zorder=2)
        self.ax.set_title('Random Numbers Histogram', fontsize=default_fontsize, fontweight='bold')
        self.ax.set_xlabel('Value', fontsize=default_fontsize)
        self.ax.set_ylabel('Frequency', fontsize=default_fontsize)
        self.canvas.draw()

    def save_plot(self):
        """Save the histogram as a PNG file."""
        file_path = filedialog.asksaveasfilename(
            defaultextension='.png',
            filetypes=[('PNG files', '*.png'), ('All Files', '*.*')],
            title='Save plot as...'
        )
        if file_path:
            self.fig.savefig(file_path)
            print(f'[INFO] Plot saved at: {file_path}')

# === Application Launch ===
if __name__ == '__main__':
    root = tk.Tk()
    app = MonochromatorApp(root)
    root.mainloop()
```

Código de Arduino para el motor.

```
// Pin definitions
const int dirPin = 2;      // Direction pin
const int stepPin = 3;     // Step pulse pin
const int stepsPerRev = 200; // Number of steps per full revolution

void setup() {
  Serial.begin(9600);  // Initialize serial communication at 9600 baud
  pinMode(dirPin, OUTPUT);
  pinMode(stepPin, OUTPUT);

  digitalWrite(dirPin, HIGH);  // Set motor direction
  Serial.println("Ready");
}

void loop() {
  // Wait for command from serial port
  if (Serial.available() > 0) {
    char command = Serial.read();

    if (command == 'S') {
      moveOneStep();            // Move the motor one step
      Serial.println("DONE");   // Notify Python that the step is completed
    }
  }
}

// Function to move the motor one step
void moveOneStep() {
  digitalWrite(stepPin, HIGH);
  delayMicroseconds(800);
  digitalWrite(stepPin, LOW);
  delayMicroseconds(800);
}
```

Código de Python para el mapa de fotoluminiscencia y ajuste de los picos.

```python
# === Import Required Libraries ===
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.colors as mcolors
import re
from io import StringIO
from scipy.ndimage import gaussian_filter1d
from scipy.optimize import curve_fit

# === Plotting Options (set True to activate) ===
plot_PL_map = False         # Option to display photoluminescence intensity map
plot_spec = False            # Option to display and fit the selected spectrum

# === File Paths (spectral data and coordinate data) ===
spectra_file = r'C:\Users\Borja\Desktop\TFG\data\Grating 150 lineas por mm\Mapas\plmap_19-6-2025_102720_515nm_50uW_30x30um_r05_02s_plmap_WSe2_ML_jun25_4K_Map.ASC'
coords_file = r'C:\Users\Borja\Desktop\TFG\data\Grating 150 lineas por mm\Mapas\plmap_19-6-2025_102720_515nm_50uW_30x30um_r05_02s_plmap_WSe2_ML_jun25_4K_Map_xy'

# === Helper Functions ===

def get_spectrum_at(x_query, y_query):
    """Return the spectrum at a specific (x, y) coordinate (in um)."""
    for i, (x, y) in enumerate(coords):
        if np.isclose(x, x_query, atol=1e-6) and np.isclose(y, y_query, atol=1e-6):
            return wavelengths, spectra[:, i]
    print(f"No point found at X={x_query}, Y={y_query}")
    return None, None

def pseudo_voigt(x, amp, cen, sigma, eta):
    """Pseudo-Voigt profile: a linear combination of Gaussian and Lorentzian profiles."""
    gaussian = amp * np.exp(-((x - cen)**2) / (2 * sigma**2))
    lorentzian = amp / (1 + ((x - cen)/sigma)**2)
    return eta * lorentzian + (1 - eta) * gaussian

```

```
def fit_peaks(wavelength, intensity, p0_list, masks):
    """Fit multiple pseudo-Voigt peaks to a given spectrum using non-linear least squares."""
    results = []
    for p0, mask in zip(p0_list, masks):
        popt, _ = curve_fit(pseudo_voigt, wavelength[mask], intensity[mask], p0=p0)
        results.append(popt)
    return results

def plot_spectrum(wavelength, intensity, peak_params, masks, labels, title):
    """Plot the original spectrum with its fitted peaks and total convolution."""
    fig, ax = plt.subplots(figsize=(6, 6), clear=True)
    ax.set_facecolor('antiquewhite')
    for spine in ax.spines.values():
        spine.set_edgecolor('tan')
        spine.set_linewidth(2)
    ax.tick_params(labelsize=15)
    ax.set_xlabel('Wavelength (nm)', fontsize=15, weight='bold')
    ax.set_ylabel('Normalized Intensity (arb. units)', fontsize=15, weight='bold')
    ax.set_xlim([650, 850])

    # Plot original spectrum
    ax.plot(wavelength, intensity, color='chocolate', label='Original Spectrum')

    # Highlight integration regions
    integration_mask = np.any(np.column_stack(masks), axis=1)
    ax.fill_between(wavelength[integration_mask], intensity[integration_mask], color='sienna', alpha=0.4, label='Integration Region')

    # Plot individual fitted peaks
    cmap = plt.get_cmap('cool')
    colors = [cmap(0.2 + i * 0.6 / (len(peak_params)-1)) for i in range(len(peak_params))]
    ideal_fit = np.zeros_like(wavelength)
    for popt in peak_params:
        ideal_fit += pseudo_voigt(wavelength, *popt)

    # Convolution to simulate spectral broadening
    sigma_conv_nm = 1.5
    step_nm = np.mean(np.diff(wavelength))
    sigma_pix = sigma_conv_nm / step_nm
    convolved_fit = gaussian_filter1d(ideal_fit, sigma=sigma_pix)

    # Rescale convolution and plot
    combined_mask = np.any(np.column_stack(masks), axis=1) & (wavelength >= 710)
    wl_fit_range = wavelength[combined_mask]
    convolved_fit_range = convolved_fit[combined_mask]
    if np.max(convolved_fit_range) > 0:
        scale_factor = np.max(intensity) / np.max(convolved_fit_range)
        convolved_fit_scaled = convolved_fit_range * scale_factor
        ax.plot(wl_fit_range, convolved_fit_scaled, c='k', lw=2.5, ls='
```

```
solid', label='Convolution', alpha=0.7)

    # Draw each fitted peak
    for popt, mask, color, label in zip(peak_params, masks, colors, labels):
        peak_cen = popt[1]
        label_with_cen = f"{label}: {peak_cen:.1f} nm"
        ax.plot(wavelength[integration_mask], pseudo_voigt(wavelength[integration_mask], *popt),
                c=color, lw=2, ls='dashed', alpha=0.9, label=label_with_cen)

    # Add energy axis (in eV)
    def wavelength_to_energy(wl_nm):
        h = 4.135667696e-15  # Planck's constant in eV*s
        c = 2.99792458e8     # Speed of light in m/s
        wl_m = wl_nm * 1e-9  # Convert nm to meters
        return (h * c) / wl_m

    ax2 = ax.twiny()
    ax2.set_xlim(ax.get_xlim())
    ticks_nm = np.linspace(*ax.get_xlim(), num=6)
    ticks_eV = wavelength_to_energy(ticks_nm)
    ax2.set_xticks(ticks_nm)
    ax2.set_xticklabels([f"{e:.2f}" for e in ticks_eV])
    ax2.set_xlabel('Energy (eV)', fontsize=15, weight='bold')
    ax2.tick_params(labelsize=15)

    # Final layout
    ax.legend(facecolor='antiquewhite', edgecolor='tan', loc='upper right', fontsize=12)
    plt.tight_layout()
    plt.show()

# === 1. Load Spectral Data (comma -> dot correction included) ===
with open(spectra_file, 'r') as f:
    lines = [line.replace(',', '.') for line in f.readlines()]
data = np.loadtxt(StringIO(''.join(lines)))
wavelengths = data[:, 0]     # First column: wavelength (nm)
spectra = data[:, 1:]        # Remaining columns: intensity at each spatial point

# === 2. Load Spatial Coordinates from Map ===
with open(coords_file, 'r') as f:
    coords_raw = f.readlines()

coords = []
for line in coords_raw:
    x_match = re.search(r'X:([-+]?\d*\.\d+|\d+)', line)
    y_match = re.search(r'Y:([-+]?\d*\.\d+|\d+)', line)
    if x_match and y_match:
        coords.append((float(x_match.group(1)), float(y_match.group(1))))
coords = np.array(coords)

# === 3. Integrate Over Selected Spectral Range ===
wl_min, wl_max = 700, 775
integration_mask = (wavelengths >= wl_min) & (wavelengths <= wl_max)
```

```
intensities = np.trapz(spectra[integration_mask, :], x=wavelengths[integration_mask], axis=0)

# Normalize intensities between 0 and 1
intensities = (intensities - np.min(intensities)) / (np.max(intensities) - np.min(intensities))

# === 4. Build PL Intensity Map ===
x_vals = np.unique(coords[:, 0])
y_vals = np.unique(coords[:, 1])
map_image = np.zeros((len(y_vals), len(x_vals)))
for i, (x, y) in enumerate(coords):
    xi = np.where(x_vals == x)[0][0]
    yi = np.where(y_vals == y)[0][0]
    map_image[yi, xi] = intensities[i]

# === 5. Plot PL Map (if activated) ===
if plot_PL_map:
    plt.figure(figsize=(6, 5))
    im = plt.imshow(map_image, extent=[x_vals[0], x_vals[-1], y_vals[0], y_vals[-1]],
                    origin='lower', aspect='auto', cmap='BrBG')
    plt.colorbar(im).ax.tick_params(labelsize=12)
    plt.xlabel(r'X ($\mu$m)', fontsize=14, weight='bold')
    plt.ylabel(r'Y ($\mu$m)', fontsize=14, weight='bold')
    plt.xticks(fontsize=12)
    plt.yticks(fontsize=12)
    plt.tight_layout()

    # Add annotation for measured point
    points = {'Measured': (30, 10.2)}
    for label, (x, y) in points.items():
        plt.plot(x, y, 'v', color='gold', markersize=8, label=label)
        plt.text(x, y + .6, label, color='gold', fontsize=12, weight='bold', style='italic')
    plt.show()

# === 6. Spectrum Analysis at Specific Point ===
x_target, y_target = 30, 10.2
wavelength, intensity = get_spectrum_at(x_target, y_target)
intensity_norm = intensity / np.max(intensity)

# === 7. Define Peak Fitting Parameters ===
# Define wavelength ranges for each peak
peak1_mask = (wavelengths >= 704) & (wavelengths <= 710)
peak2_mask = (wavelengths >= 717.5) & (wavelengths <= 720.66)
peak3_mask = (wavelengths >= 724) & (wavelengths <= 730.6)
peak4_mask = (wavelengths >= 730.65) & (wavelengths <= 734.3)
peak5_mask = (wavelengths >= 735.7) & (wavelengths <= 739.5)
peak7_mask = (wavelengths >= 743.7) & (wavelengths <= 749)
peak8_mask = (wavelengths >= 754.49) & (wavelengths <= 780)
all_peak_masks = [peak1_mask, peak2_mask, peak3_mask, peak4_mask, peak5_mask, peak7_mask, peak8_mask]

# Labels for each transition
peak_labels = ['Neutral Exciton', 'Trion', 'Localized State 1', 'Localized State 2',
               'Localized State 3', 'Localized State 4', 'Localized
```

```
    State 5']

# Initial parameter guesses: [amplitude, center, width, eta]
target_p0 = [
    [0.52, 705, 4, 0.5],
    [0.33, 719.9, 1, 0.5],
    [0.99, 728.7, 1, 0],
    [1.00, 732.6, 2, 0.5],
    [0.90, 736.3, 1, 0.5],
    [0.77, 743.7, 1, 0],
    [0.1, 755, 1, 0],
]

# === 8. Fit and Plot Spectrum (if enabled) ===
if plot_spec:
    target_peaks = fit_peaks(wavelength, intensity_norm, target_p0,
    all_peak_masks)
    plot_spectrum(wavelength, intensity_norm, target_peaks,
    all_peak_masks, peak_labels, title='Measured Point')
```

Código de Python para el estudio de la fotoluminiscencia con la potencia de excitación.

```python
# === Libraries ===
import os
import numpy as np
import matplotlib.pyplot as plt
import re
import scipy.constants as const
from scipy.optimize import curve_fit

# === Configuration ===
folder = r"C:\Users\Borja\Desktop\TFG\data\Grating 600 lineas por mm\Estudio potencia"
files = [f for f in os.listdir(folder) if f.endswith(".asc")]

# Peak positions (in nm), corrected to align with 712.35 nm
wvs = np.array([706, 721, 729.1, 733.2, 738.5, 747, 755])
wvs = wvs + (712.35 - wvs[0])  # Reference shift

# Extract power from filename (in uW)
def extract_power(filename):
    match = re.search(r"([\d\.]+)uW", filename)
    return float(match.group(1)) if match else 0

files = sorted(files, key=extract_power)
colors = plt.cm.viridis(np.linspace(0, 1, len(wvs)))
integrated_intensities = {wv: [] for wv in wvs}
powers = []

# nm -> eV conversion
def wavelength_to_eV(wavelength_nm):
    return (const.h * const.c) / (wavelength_nm * 1e-9 * const.e)

# Peak labels
peak_labels = [r"$X^0$", r"$X^-$", r"$L_1$", r"$L_2$", r"$L_3$", r"$L_4$", r"$L_5$"]

# === Stacked Plot ===
face_color, edge_color = 'floralwhite', 'wheat'
plt.figure(figsize=(6, 6))
ax = plt.gca()
ax.set_facecolor(face_color)
for spine in ax.spines.values():
    spine.set_edgecolor(edge_color)
```

```
    spine.set_linewidth(1.5)

offset = 0.5  # Vertical offset for stacked display

for i, filename in enumerate(files):
    filepath = os.path.join(folder, filename)
    with open(filepath, 'r', encoding='utf-8') as f:
        lines = f.readlines()

    data = []
    for line in lines:
        line = line.replace(',', '.').strip()
        parts = line.split()
        if len(parts) == 2:
            try:
                x = float(parts[0])
                y = float(parts[1])
                data.append([x, y])
            except ValueError:
                continue

    data = np.array(data)
    if data.size == 0:
        continue

    x = data[:, 0]
    y = data[:, 1]
    power_val = extract_power(filename)
    powers.append(power_val)

    # Plot spectrum with vertical offset
    ax.plot(x, y + i * offset, label=f"{power_val:.1f} uW", c=colors
   [::-1][i])

    # Integration around each peak (+-1.8 nm)
    for wv in wvs:
        mask = (x >= wv - 1.8) & (x <= wv + 1.8)
        area = np.trapz(y[mask], x[mask])
        integrated_intensities[wv].append(area)
        ax.fill_between(x[mask], y[mask] + i * offset, i * offset,
                        alpha=0.6, color='wheat')

# Label peaks on the highest power spectrum (i=0)
for wv, label in zip(wvs, peak_labels):
    ax.text(wv, 1.2e4, label, ha='center', va='bottom',
            fontsize=12, weight='bold', color='saddlebrown')

ax.set_xlabel("Wavelength (nm)", fontsize=15, weight='bold')
ax.set_ylabel("Intensity (arb. units)", fontsize=15, weight='bold')
ax.legend(title="Excitation Power", fontsize=12, loc='upper left',
          edgecolor=edge_color, facecolor=face_color)
ax.tick_params(axis='both', labelsize=11)

# Top axis: Energy in eV
def nm_to_ev_ticks(x): return wavelength_to_eV(x)
def ev_to_nm_ticks(x): return (const.h * const.c) / (x * const.e) * 1e9

secax = ax.secondary_xaxis('top', functions=(nm_to_ev_ticks,
```

```
    ev_to_nm_ticks))
secax.set_xlabel('Energy (eV)', fontsize=15, weight='bold')
secax.tick_params(axis='x', labelsize=11)

plt.tight_layout()
plt.show()

# === Log-Log Fit Excluding First Three Powers ===

powers = np.array(powers)
log_powers = np.log10(powers)

plt.figure(figsize=(6, 6))
ax2 = plt.gca()
ax2.set_facecolor(face_color)
for spine in ax2.spines.values():
    spine.set_edgecolor(edge_color)
    spine.set_linewidth(1.5)

vertical_offset = 1.2  # Visual y-axis offset

for i, (wv, label) in enumerate(zip(wvs, peak_labels)):
    intensities = np.array(integrated_intensities[wv])
    log_intensities = np.log10(intensities)

    # Exclude first three points (low power region)
    x_fit = log_powers[3:]
    y_fit = log_intensities[3:]

    # Fit model: log I = alpha * log P + log A
    def linear_model(x, alpha, logA):
        return alpha * x + logA

    popt, pcov = curve_fit(linear_model, x_fit, y_fit)
    alpha, logA = popt
    alpha_err = np.sqrt(np.diag(pcov))[0]

    x_plot = np.linspace(log_powers.min(), log_powers.max(), 100)
    y_plot = linear_model(x_plot, *popt)

    # Offset each curve for better visibility
    y_offset = i * vertical_offset

    # Excluded points (hollow markers)
    ax2.plot(log_powers[:3], log_intensities[:3] + y_offset, 'o',
             markerfacecolor='none', markeredgecolor=colors[i])

    # Fitted points (solid)
    ax2.plot(log_powers[3:], log_intensities[3:] + y_offset, 'o',
             color=colors[i],
             label=f"{label}: alpha = {alpha:.2f} +- {alpha_err:.2f}")

    # Fitted line
    ax2.plot(x_plot, y_plot + y_offset, '-', color=colors[i])

ax2.set_xlabel("log(Excitation Power (uW))", fontsize=15, weight='bold'
    )
ax2.set_ylabel("log(Integrated Intensity (arb. units))", fontsize=15,
```

```
    weight='bold')
ax2.legend(fontsize=12, loc='upper left', edgecolor=edge_color,
    facecolor=face_color)
ax2.tick_params(axis='both', labelsize=11)

plt.tight_layout()
plt.show()
```

Código de Python para el estudio de la fotoluminiscencia con la temperatura.

```python
import os
import numpy as np
import matplotlib.pyplot as plt
import re
import scipy.constants as const
from scipy.signal import find_peaks
from scipy.interpolate import UnivariateSpline

# === Physical Constants ===
h = const.Planck  # Planck constant (J*s)
c = const.c       # Speed of light in vacuum (m/s)
e = const.e       # Elementary charge (C)

# === File and Folder Setup ===
folder = r"C:\Users\Borja\Desktop\TFG\data\Grating 600 lineas por mm\Estudio temperatura"
files = [f for f in os.listdir(folder) if f.lower().endswith(".asc")]  # List .asc files

# === Labels for spectral peaks in the lowest-temperature spectrum ===
peak_labels = [r"$X^0$", r"$X^-$", r"$L_1$", r"$L_2$", r"$L_3$", r"$L_4$", r"$L_5$"]
wvs = np.array([705, 721, 729.1, 733.2, 738.5, 747, 755])
wvs = wvs + (712.35 - wvs[0])  # Align labels using X0 as reference

def extract_temperature(filename):
    """Extract temperature in Kelvin from filename (e.g., '77K' -> 77).
    """
    match = re.search(r"(\d+)[Kk]", filename)
    return int(match.group(1)) if match else 0

def varshni_wavelength(T, E0=1.75, alpha=4.5e-4, beta=170):
    """Compute bandgap wavelength (in nm) using the Varshni equation.
    """
    E_T = E0 - (alpha * T**2) / (T + beta)  # Bandgap energy in eV
    lambda_nm = (h * c / (E_T * e)) * 1e9   # Convert to wavelength (nm)
    return lambda_nm

# Sort files by extracted temperature
files = sorted(files, key=extract_temperature)
```

```python

# === Initialize Plot ===
plt.figure(figsize=(6, 6))
offset = .5  # Vertical offset between spectra
colors = plt.cm.winter(np.linspace(0, 1, len(files)))  # Color map by temperature

exciton_peaks = []
exciton_heights = []
temperatures = []

# === Process Each Spectrum ===
for i, filename in enumerate(files):
    filepath = os.path.join(folder, filename)

    # Read file content
    with open(filepath, 'r', encoding='utf-8') as f:
        lines = f.readlines()

    # Parse data lines (wavelength, intensity)
    data = []
    for line in lines:
        line = line.replace(',', '.').strip()
        parts = line.split()
        if len(parts) == 2:
            try:
                x = float(parts[0])
                y = float(parts[1])
                data.append([x, y])
            except ValueError:
                continue

    data = np.array(data)
    if data.size == 0:
        print(f"[!] Empty or unreadable file: {filename}")
        continue

    x = data[:, 0]
    y = data[:, 1]
    y_norm = y / np.max(y)  # Normalize intensity

    T = extract_temperature(filename)
    temperatures.append(T)

    # Estimate exciton region using Varshni model
    lambda_center = varshni_wavelength(T)
    delta = 6  # Search range in nm
    search_min = lambda_center - delta
    search_max = lambda_center + delta

    # Detect peaks
    peaks, properties = find_peaks(y_norm, height=0.3)
    candidate_peaks = [(x[peak], y_norm[peak]) for peak in peaks if search_min <= x[peak] <= search_max]

    if candidate_peaks:
        peak_wavelength, peak_height = max(candidate_peaks, key=lambda p: p[1])
```

```
        exciton_peaks.append(peak_wavelength)
        exciton_heights.append(peak_height)
        print(f"[INFO] T = {T} K, exciton peak = {peak_wavelength:.2f} nm")
    else:
        print(f"[WARN] No exciton peak found between 700-760 nm in {filename}")
        exciton_peaks.append(np.nan)
        exciton_heights.append(np.nan)

    # Plot the spectrum with vertical offset
    plt.plot(x, y_norm + i * offset, color=colors[i])

    # Annotate temperature label
    label_x = 702
    label_y = y_norm[x >= label_x][0] + i * offset + 0.03
    plt.text(label_x, label_y, f"{T} K", color=colors[i], fontsize=7.5,
         ha='left', va='bottom', fontweight='bold',
         bbox=dict(facecolor=colors[i], alpha=0.3, edgecolor='none', boxstyle='round,pad=0.2'))

# === Plot Formatting ===
ax = plt.gca()
face_color = 'azure'
border_color = 'slateblue'
ax.set_facecolor(face_color)

ax.set_xlabel("Wavelength (nm)", fontsize=14, fontweight='bold')
ax.set_yticklabels([])
ax.tick_params(axis='y', which='both', length=0)
ax.tick_params(axis='x', which='major', labelsize=12, width=2,
               color=border_color, direction='inout', length=6)

for spine in ax.spines.values():
    spine.set_linewidth(2)
    spine.set_color(border_color)

# === Add Secondary X Axis: Energy in eV ===
def nm_to_ev(x_nm):
    return (h * c / (x_nm * 1e-9)) / e

def ev_to_nm(E_eV):
    return (h * c / (E_eV * e)) * 1e9

secax = ax.secondary_xaxis('top', functions=(nm_to_ev, ev_to_nm))
secax.set_xlabel('Energy (eV)', fontsize=14, fontweight='bold')
secax.tick_params(axis='x', which='major', labelsize=12, width=2,
                  color=border_color, direction='inout', length=6)

# === Plot Exciton Peaks ===
valid_indices = ~np.isnan(exciton_peaks)
valid_peaks = np.array(exciton_peaks)[valid_indices]
valid_heights = np.array(exciton_heights)[valid_indices]
valid_i = np.arange(len(files))[valid_indices]

plt.plot(valid_peaks, valid_heights + valid_i * offset,
         color='slateblue', ls='None', marker='o', markersize=5, zorder=3)
```

```python

# === Optional: Smooth interpolation ===
if len(valid_peaks) >= 4:
    spline = UnivariateSpline(valid_peaks, valid_heights + valid_i * offset, s=150)
    x_smooth = np.linspace(min(valid_peaks), max(valid_peaks), 600)
    y_smooth = spline(x_smooth)
    plt.plot(x_smooth, y_smooth, color='slateblue', linestyle='--', linewidth=1.5, zorder=2)
else:
    print("[WARN] Smooth interpolation skipped: insufficient valid data points.")

# Set plot x-range
ax.set_xlim([700, 765])

# === Add Peak Labels on the First Spectrum ===
first_filepath = os.path.join(folder, files[0])
with open(first_filepath, 'r', encoding='utf-8') as f:
    lines = f.readlines()

data = []
for line in lines:
    line = line.replace(',', '.').strip()
    parts = line.split()
    if len(parts) == 2:
        try:
            x_val = float(parts[0])
            y_val = float(parts[1])
            data.append([x_val, y_val])
        except ValueError:
            continue

data = np.array(data)
x_data = data[:, 0]
y_data = data[:, 1]
y_norm_data = y_data / np.max(y_data)

for wl, label in zip(wvs, peak_labels):
    idx = (np.abs(x_data - wl)).argmin()
    y_val = y_norm_data[idx]
    plt.text(wl, y_val + 0.3, label, fontsize=13, ha='center', va='bottom',
         color='navy', fontweight='bold',
         bbox=dict(facecolor='whitesmoke', alpha=0.9, edgecolor='none', boxstyle='round,pad=0.2'))

# Layout and show
plt.tight_layout()
plt.show()

# === Save Peak Results ===
output_file = os.path.join(folder, "excitonic_peaks.txt")

with open(output_file, 'w', encoding='utf-8') as f:
    f.write("Temperature (K)\tWavelength (nm)\n")
    for T, peak in zip(temperatures, exciton_peaks):
        if not np.isnan(peak):
```

```
            f.write(f"{T}\t{peak:.2f}\n")
        else:
            f.write(f"{T}\tNaN\n")

print(f"[INFO] Output file saved: {output_file}")
```

Código de Python para el ajuste a la ecuación de Varshni.

```python
import numpy as np
import matplotlib.pyplot as plt
from scipy.odr import ODR, Model, RealData
import scipy.constants as const
import os

# === Physical Constants ===
h = const.Planck    # Planck constant (J*s)
c = const.c         # Speed of light (m/s)
e = const.e         # Elementary charge (C)

def nm_to_ev(wavelength_nm):
    """
    Convert wavelength from nanometers to electronvolts.
    """
    return (h * c / (wavelength_nm * 1e-9)) / e

# === Varshni Model for ODR Fitting ===
def varshni_odr(B, T):
    """
    Varshni equation: describes the temperature dependence of the semiconductor bandgap.
    B: [E0, alpha, beta] are the model parameters.
    T: temperature in Kelvin.
    Returns E(T) in eV.
    """
    E0, alpha, beta = B
    return E0 - (alpha * T**2) / (T + beta)

# === Load Experimental Excitonic Peaks Data ===
folder = r"C:\Users\Borja\Desktop\TFG\data\Grating 600 lineas por mm\Estudio temperatura"
filename = os.path.join(folder, "excitonic_peaks.txt")

temperatures = []
wavelengths = []

# Read two-column data: temperature (K) and excitonic peak wavelength (nm)
with open(filename, 'r', encoding='utf-8') as f:
    next(f)  # Skip header line
    for line in f:
        parts = line.strip().split()
        if len(parts) == 2:
```

```
            T = float(parts[0])
            try:
                wl = float(parts[1])
                temperatures.append(T)
                wavelengths.append(wl)
            except ValueError:
                continue  # Skip lines with invalid values

temperatures = np.array(temperatures)
wavelengths = np.array(wavelengths)
energies = nm_to_ev(wavelengths)  # Convert wavelength to energy (eV)

# === Orthogonal Distance Regression (ODR) ===
# Define measurement uncertainties
T_error = 0.5     # Temperature uncertainty in K
y_error = 0.005   # Energy uncertainty in eV

# Prepare data and model for ODR
model = Model(varshni_odr)
data = RealData(temperatures, energies, sx=T_error, sy=y_error)
odr = ODR(data, model, beta0=[1.75, 4e-4, 200])  # Initial parameter
    guesses
output = odr.run()

# Extract fitted parameters and standard deviations
E0, alpha, beta = output.beta
dE0, dalpha, dbeta = output.sd_beta

# Generate smooth fit curve over temperature range
T_fit = np.linspace(min(temperatures), max(temperatures), 200)
E_fit = varshni_odr([E0, alpha, beta], T_fit)

# === Manual Calculation of Coefficient of Determination (R^2) ===
residuals = energies - varshni_odr([E0, alpha, beta], temperatures)
ss_res = np.sum(residuals**2)
ss_tot = np.sum((energies - np.mean(energies))**2)
r_squared = 1 - ss_res / ss_tot

# === Stylized Plot ===
face_color, edge_color = 'thistle', 'plum'
plt.figure(figsize=(6, 5))
ax = plt.gca()
ax.set_facecolor(face_color)
for spine in ax.spines.values():
    spine.set_edgecolor(edge_color)
    spine.set_linewidth(2)

# Plot experimental data with error bars
ax.errorbar(
    temperatures, energies, xerr=T_error, yerr=y_error,
    fmt='o', color='slateblue', ecolor='navy', capsize=3,
    label="Excitonic peaks", zorder=3, mec='k'
)

# Plot the Varshni model fit
ax.plot(
    T_fit, E_fit, '--', color='rebeccapurple', linewidth=2, alpha=0.6,
    label=r"$E(T) = E_0 - \frac{\alpha T^2}{T + \beta}$"
```

```
)

# Labels and legend
ax.set_xlabel("Temperature (K)", fontsize=14, weight='bold')
ax.set_ylabel("Excitonic energy (eV)", fontsize=14, weight='bold')
ax.tick_params(axis='both', labelsize=12)
ax.legend(fontsize=13, loc='best', edgecolor=edge_color, facecolor=
    face_color)

plt.tight_layout()
plt.show()

# === Display Fit Results ===
print("[ODR FIT RESULTS]")
print(f"E0      = {E0:.4f} +- {dE0:.4f} eV")
print(f"alpha   = {alpha:.2e} +- {dalpha:.2e} eV/K")
print(f"beta    = {beta:.2f} +- {dbeta:.2f} K")
print(f"Coefficient of determination R^2 = {r_squared:.4f}")
```